Security

Zephyr RTOS Course



Objectives

- Why are we having this lesson
- Go over what Zephyr offers in terms of security
- Dive specifically onto secure boot



Format



- Lecture with **live demo**
- Homework after each session (checked by us)
- Recordings will be shared
- Questions addressed at the end in a **Q&A session**

Today's presentation

First half

- Why security became mandatory
- What Zephyr gives you
- What it does **not** give you

Second half

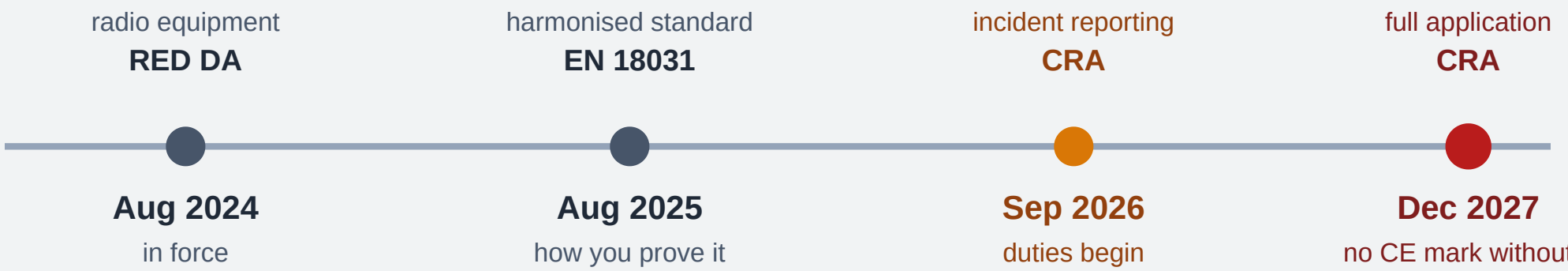
- Secure boot, and how the chain works
- Wiring it into Zephyr
- Live demo on QEMU: valid, tampered and wrong-key images



Why a Security Lesson?

EU Regulation Timeline

EU rules for connected products



A product designed now ships under the 2027 rules

i NOTE

Elsewhere: the US has the FCC's **Cyber Trust Mark**. It's voluntary, built on NIST IR 8259A/B, and still rolling out administrator applications in 2026. Unlike the EU rules, it is not mandatory.

What the Regulations Actually Ask For

Zephyr®

They describe what the product must achieve and leave the implementation to you:

- **No universal default passwords**: per-device credentials
- **A way to ship security updates**, and to prove the device only accepts yours
- **Vulnerability handling**: a contact point, a process, a disclosure policy
- **An inventory of what is inside the product**: SBOM
- **Protection of sensitive data at rest**: keys, credentials, personal data

> i NOTE
>
> **SBOM (Software Bill of Materials):** a machine-readable list of every software component in the firmware, with its version, supplier and license. It lets you check quickly whether a newly published vulnerability affects your product.



What "Secure" Means for a Device



There are four properties, and every security feature protects one or more of them:

- **Integrity**: the code and data are what you put there
- **Authenticity**: you can tell who produced the code, the data, the device
- **Confidentiality**: secrets stay secret, including keys, credentials, user data and your IP
- **Availability**: the device keeps working, and can be recovered when it does not



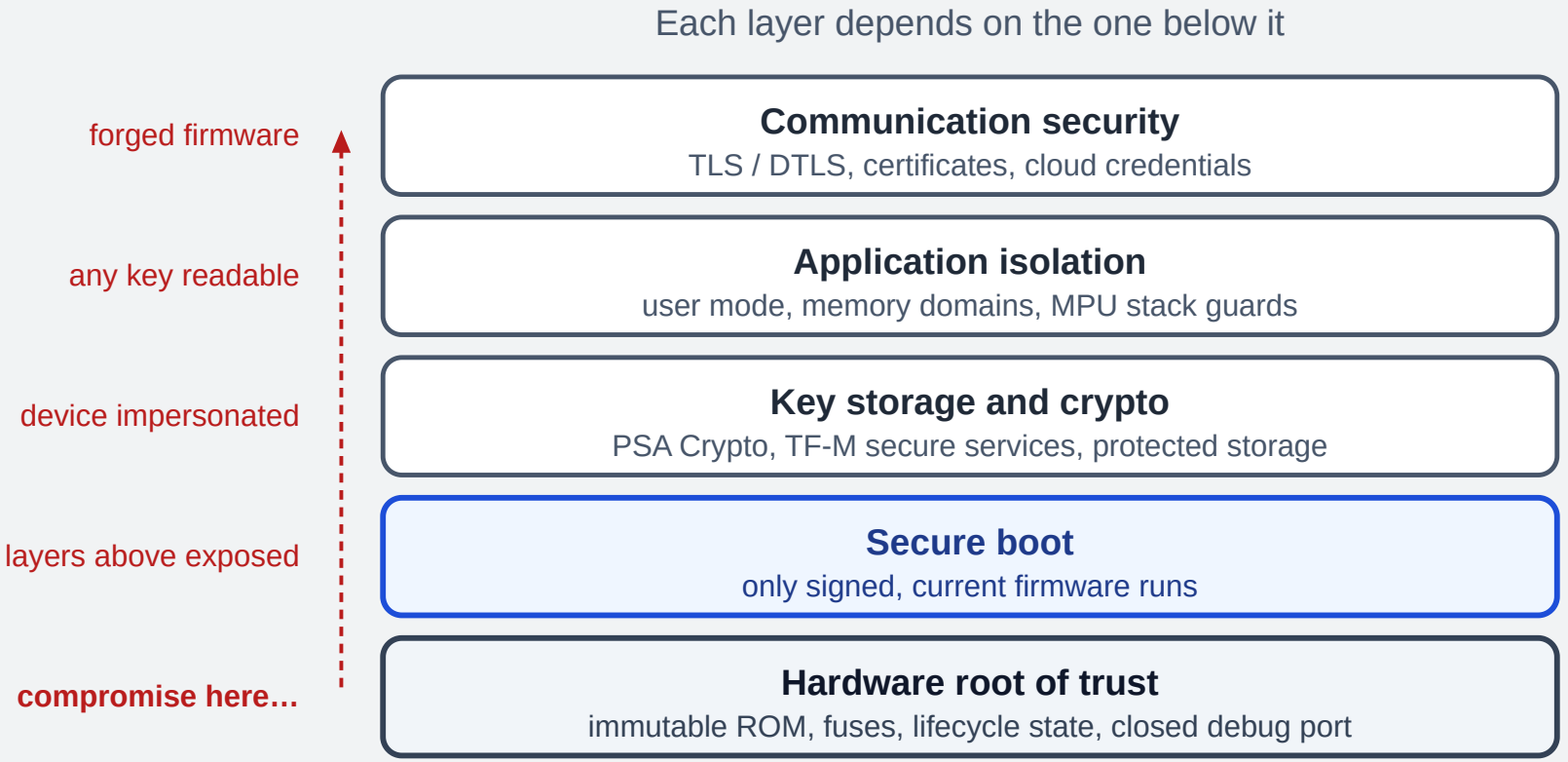
Some Secure Practices



For a connected embedded product, in rough order of what a review will check:

- **Verify code before running it**: signed images, checked at every boot
- **Update securely**: signed, rollback-protected, recoverable when it fails
- **Store keys where software cannot read them**: not hardcoded in a header or stored in plain flash
- **Least privilege**: code reaches only what it needs
- **No shared secrets across the fleet**: per-device credentials
- **Disable development features before shipping**: debug ports, shells, logs
- **Know what is in the build, and track its known vulnerabilities**: SBOM plus a process

How the Layers Depend on Each Other





Security in Zephyr



Default isolation level

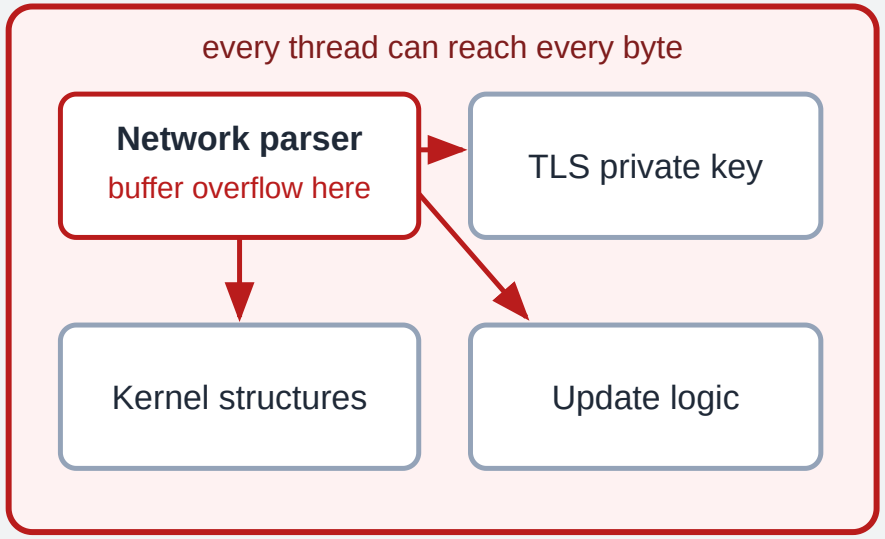


By default every thread can read and write every byte: your TLS key, the kernel's structures, the update logic.

- A firmware image is one flat address space
- A bug in the network input parser can read or corrupt any memory in the system
- Code that handles untrusted input runs with the same privileges as the kernel

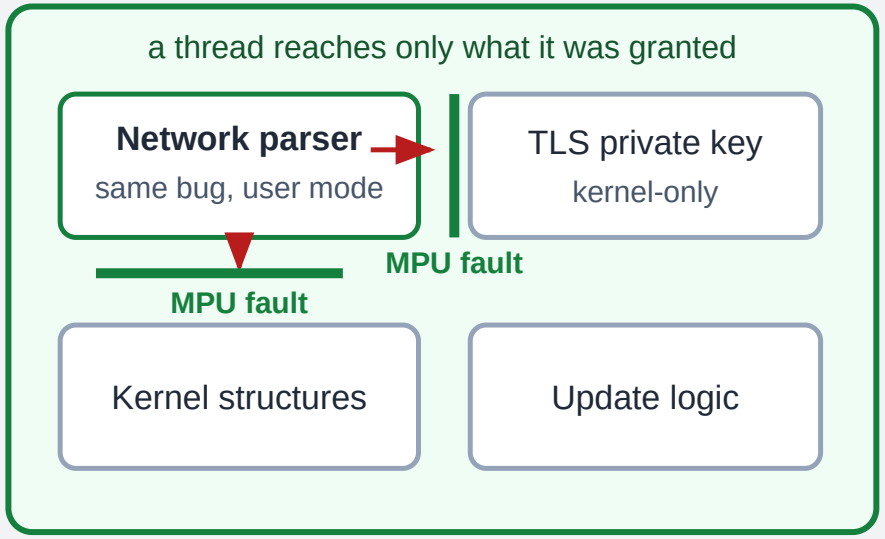
With and Without Isolation

No isolation: one privilege level



A bug here reaches the key,
the kernel and the updater

User mode + memory domains



The same bug faults the parser thread
and the kernel decides what happens next

Isolation Features in the Kernel

- **User mode threads** (`CONFIG_USERSPACE`): threads run unprivileged; the kernel treats them as untrusted, and they reach the kernel only through system calls
- **Memory domains**: explicit, per-thread memory access; anything not granted faults
- **MPU stack guard** (`CONFIG_MPU_STACK_GUARD`): an unmapped region below the stack, so an overflow faults instead of quietly corrupting the neighbour
- **Stack sentinel** (`CONFIG_STACK_SENTINEL`): canary value checked on context switch
- **Hardware stack protection** (`CONFIG_HW_STACK_PROTECTION`)

> **i NOTE**
>
> All of this is opt-in, and most of it needs an **MPU**: the hardware block that enforces which regions a thread may access. A default `prj.conf` enables none of it.

Crypto and Keys

- **PSA Crypto API**: Arm's portable interface for crypto and **key storage**
  - In Zephyr, implemented on top of **Mbed TLS**
  - Keys are referenced by **handle**, not by pointer to a buffer
- **TF-M**: on Cortex-M33/M55, crypto and key storage run in the **secure world**; the application asks for operations, never for the key
- **TLS / DTLS** for transport, with the same key-handle model

> **NOTE**
> The intent is that application code can use a key without being able to read it.

Vulnerability Handling and Process

- A **Security Working Group** and a documented **vulnerability reporting** path (`vulnerabilities@zephyrproject.org`), with embargo and CVE handling
- Published **secure coding guidelines** and threat models in the Zephyr docs
- A mapping of Zephyr features to **ETSI EN 303 645**, the consumer IoT baseline
- The **hardening tool**, which you can run on your own build today

> **NOTE**
>
> The CRA requires a vulnerability handling process. Being upstream means you can point at this one instead of writing your own.



Checking a Build: `hardenconfig`




```
west build -b <board> samples/hello_world -t hardenconfig
```

```
                name                        |  current   | recommended || check result
============================================================================================
CONFIG_BOOT_BANNER                          |     y      |      n      ||     FAIL
CONFIG_BUILD_OUTPUT_STRIPPED                |     n      |      y      ||     FAIL
CONFIG_HW_STACK_PROTECTION                  |     n      |      y      ||     FAIL
CONFIG_MPU_STACK_GUARD                      |     n      |      y      ||     FAIL
CONFIG_STACK_SENTINEL                       |     n      |      y      ||     FAIL
CONFIG_PRINTK                               |     y      |      n      ||     FAIL
```

It compares your build against the Security WG's recommended settings.

Mapping Those Practices onto Zephyr

Practice	What Zephyr provides
Verify code before running it	**MCUboot**, in-tree, plus `west sign`
Update securely	MCUboot swap and revert, downgrade prevention; SMP/DFU, LwM2M, hawkBit
Store keys out of software's reach	**PSA Crypto** key handles on Mbed TLS; **TF-M** on Cortex-M33/M55
Least privilege	`CONFIG_USERSPACE`, memory domains, MPU stack guard
Per-device credentials	TLS/DTLS consumes them, but **generating and provisioning them is yours**
Disable development features	`hardenconfig` flags shell, logs, banner, debug options
Know the build and its CVEs	`west spdx` for SBOM; Security WG advisories

Effort Compared: Zephyr vs FreeRTOS

Task	FreeRTOS	Zephyr
Verified/signed boot	Pick and integrate a bootloader per SoC; official ports are lab-grade	In-tree **MCUboot**, one sysbuild flag
Memory isolation	Per-architecture MPU port, wired up by you	`CONFIG_USERSPACE` + memory domains, Kconfig
Crypto & key storage	Assemble Mbed TLS / PKCS#11 / vendor SDK yourself	**PSA Crypto** in-tree; **TF-M** on Cortex-M33/M55
Secure update	Assemble an OTA agent, bootloader and cloud service	MCUboot swap/revert + SMP/DFU, LwM2M, hawkBit, in-tree
Config auditing	Manual review	`west build -t hardenconfig` and initiatives to reach further
SBOM	Your own process	`west spdx`
Vulnerability process	Per-vendor, ad hoc	Security WG, published CVE process

i NOTE

Both have similar building blocks. The difference is integration: Zephyr's are in-tree, wired together by Kconfig and sysbuild. FreeRTOS's are separate libraries and vendor or reference integrations you assemble per platform.



Outside Zephyr's Scope



- **The hardware root of trust**: the silicon vendor's, and support varies by platform
- **The key process**: generating, storing and rotating the signing key
- **The threat model**: what your product is worth to an attacker



Where Secure Boot Fits



Secure boot is the bottom layer. The others depend on it:

- Isolation separates threads inside the firmware you chose
- Key storage protects secrets from the firmware you chose
- TLS authenticates the device as whatever firmware is running



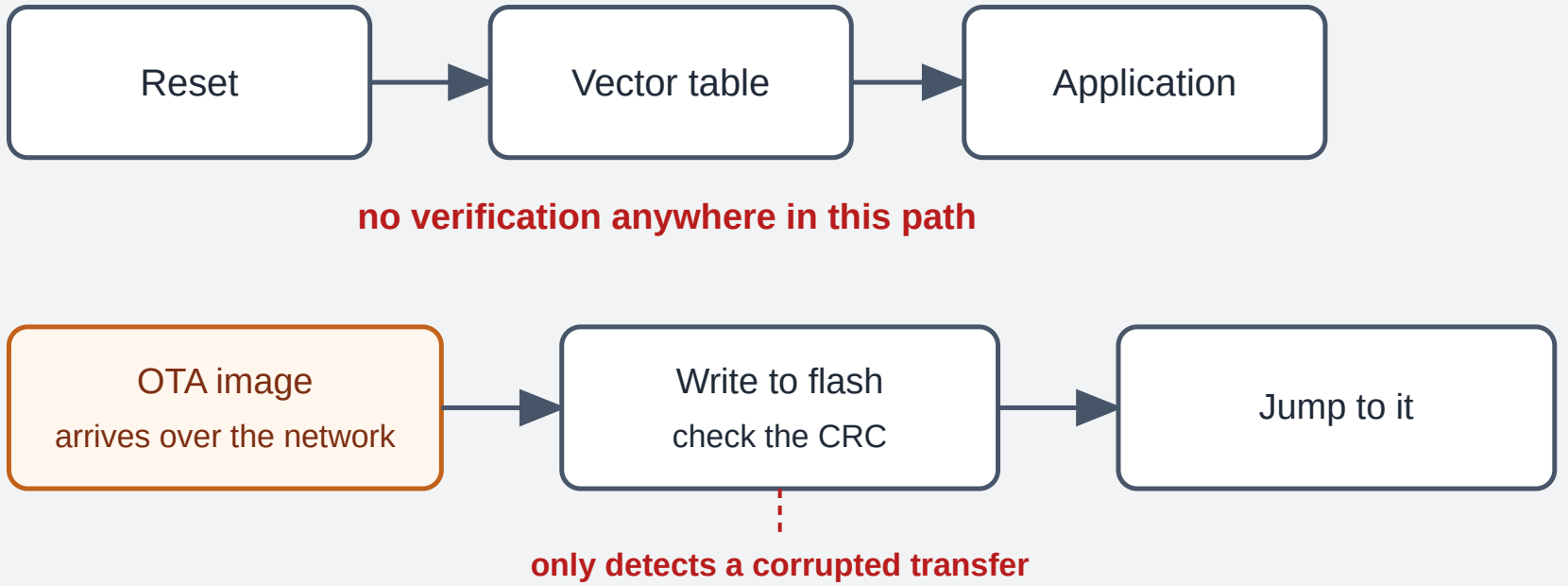
The "Traditional" Boot Flow

CRC, Hash, Signature, Encryption

Mechanism	Provides	Typical stage	Key on device
CRC	Detects accidental corruption only; anyone can recompute it	Traditional boot	None
Hash	Integrity against one exact build, if the reference hash cannot be changed	Immutable first stage	None
Signature	Integrity + authenticity	Any stage, including updatable ones	Public (verify only)
Encryption	Confidentiality	Any stage holding IP or secrets	Secret (decrypt)

> **NOTE**
>
> **What a signed image looks like** (MCUboot): `[ header | firmware | SHA256 · PUBKEY · ECDSASIG ]`
> Build machine: hash the image, sign the hash with the private key, append both.
> Device: recompute the hash, check the signature with the public key it trusts.

How Most Devices Boot



Trust model: whoever can write flash controls the device



Why This Matters More Now



- **Fleet scale**: one working exploit applies to every unit
- **Remote update is mandatory**: the update channel is also an entry point for attackers
- **10+ year lifetimes**: new CVEs keep appearing for years after the product ships



Secure Boot

Definition

"Secure boot" is a broad term without a single technical definition.

- No standard fixes its meaning
- Each silicon vendor decides what it calls secure boot
- The same words on two datasheets can describe different mechanisms

OTHER NAMES USED FOR THIS AREA

verified boot · high-assurance boot · trusted boot

measured boot · authenticated boot · encrypted boot

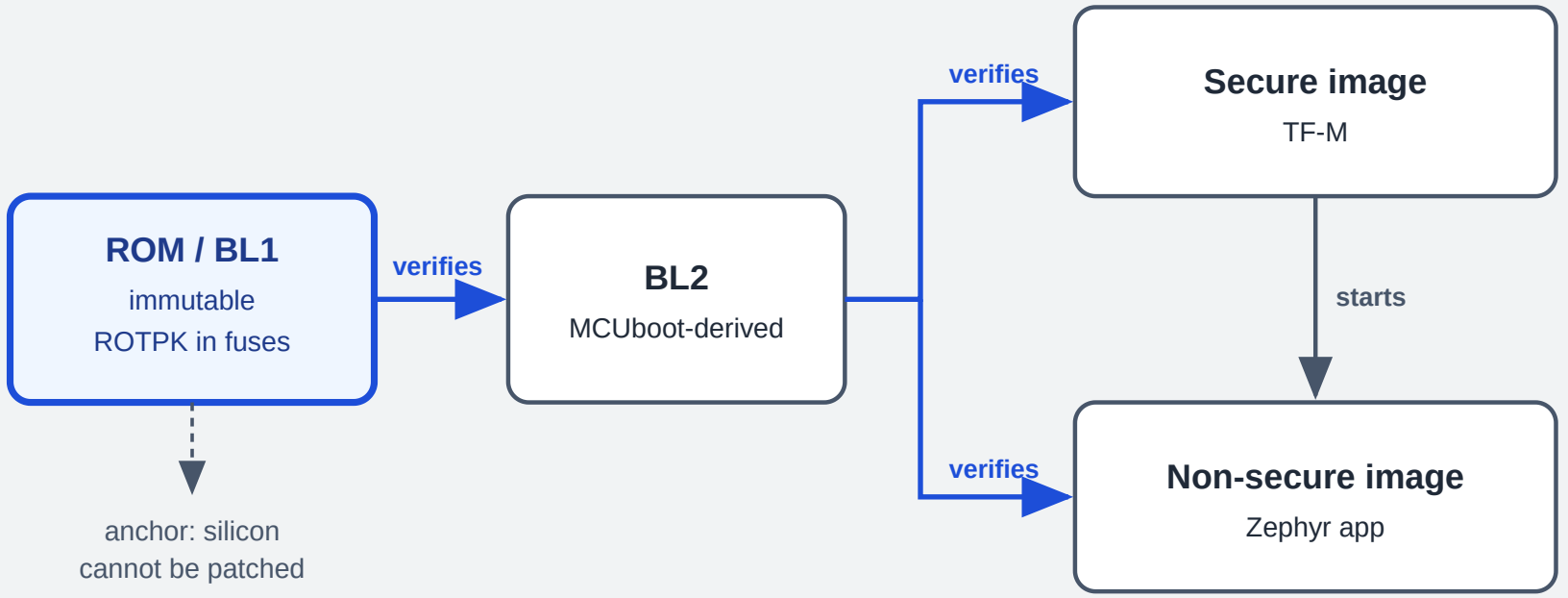


Some Boot Terms



Term	How it works	If the firmware was tampered with
Verified / authenticated boot (NXP: High Assurance Boot)	Each stage checks the next image's signature with a public key it holds. No TPM needed; the key is safe to leak, but must not be replaceable	Stops the boot
Trusted boot	No fixed meaning. Arm TF-A: a signature chain, i.e. verified boot. Intel tboot: only records, i.e. measured boot	Depends on the vendor
Measured boot	Each stage stores a hash of the next in a TPM, which can be added to but not erased. A server checks the signed record later	Boots, but a server can refuse the device
Encrypted boot	Firmware stored encrypted; the bootloader decrypts it on each boot with a secret key kept on the device	Boots: hides the firmware, does not check it

Typical Fully Featured Secure Boot Flow



BL2 verifies both images before either one runs

> **i NOTE**
>
> **ROTPK** (Root Of Trust Public Key): the public key held at the anchor



How secure is my boot process?



The first three rows build on each other. The last two apply at any level.

Threat	Possible when	Defense	Still defeated by
Malicious OTA update	The bootloader boots whatever is in the slot	Signature check; public key built into the bootloader	Anyone who can write flash: debug port, a firmware bug
Overwriting the bootloader or its key	Bootloader in writable flash, debug port open	Flash and debug port locked	Erasing the whole chip, then flashing any firmware
Erase-and-reflash	Nothing below the bootloader checks it	ROM checks the bootloader's signature; key hash in OTP	ROM bugs, fault injection (glitching)
Reinstalling an older, vulnerable signed image	The bootloader accepts any version with a valid signature	**Downgrade prevention**: version checked against a counter in OTP	Counter stored in memory an attacker can reset
An update that fails and bricks the device	One image slot: the update overwrites the working image	**Rollback**: previous image kept, restored if the new one never confirms	App confirms itself (`image_ok`) before it's really working



Secure Boot in Zephyr

MCUboot

Zephyr's in-tree bootloader.

- Verifies the image signature before jumping to it
- Handles the update itself: swap, and **revert** if the new image never confirms
- Supports downgrade prevention via a monotonic security counter
- Also the base for TF-M's BL2, on Cortex-M33/M55

> **i NOTE**
> MCUboot is a concrete implementation of the chain shown in the "typical flow" slide.



Sysbuild



Zephyr's multi-image build system.

- One `west build` produces the app **and** everything it depends on to boot: MCUboot, a second core's image, a TF-M secure image
- Each image is configured, built and, for MCUboot, **signed** as part of the same build graph
- Without it, each image needs its own build directory, key setup and flashing step



Wiring MCUboot In With Sysbuild




```
# sysbuild.conf
SB_CONFIG_BOOTLOADER_MCUBOOT=y
SB_CONFIG_BOOT_SIGNATURE_KEY_FILE="keys/my-signing-key.pem"
```

- The key file propagates to **both** images: `CONFIG_MCUBOOT_SIGNATURE_KEY_FILE` signs the app, `CONFIG_BOOT_SIGNATURE_KEY_FILE` embeds the matching public key in MCUboot itself
- Signing happens automatically at build time, with no separate `imgtool` step
- The default key is a public, insecure development key checked into MCUboot. It is fine for this demo, but a product needs its own private key.



With TF-M: BL2 Instead of Sysbuild



On boards built with TF-M, the bootloader is TF-M's BL2, which is based on MCUboot.
It is built without sysbuild. The demo uses this setup.

```
# prj.conf
CONFIG_BUILD_WITH_TFM=y
CONFIG_TFM_KEY_FILE_S="keys/secure.pem"
CONFIG_TFM_KEY_FILE_NS="keys/non-secure.pem"
CONFIG_TFM_IMAGE_VERSION_NS="0.0.1+0"
```

- `west build` also builds TF-M, signs the images and merges them into `tfm_merged.hex`
- Each image has its own key: one for TF-M, one for the app
- If no keys are set, TF-M's development keys are used
- The version is also used as the anti-rollback counter

The Demo: Why QEMU

- No hardware needed, fast to iterate, fully scriptable: every step in this demo is a repeatable command, not a manual flash
- Same verification logic that runs on silicon: signature checks, rejection, swap and revert all behave identically
- **Why the AN521 board**: Arm's public Cortex-M33 reference design, so QEMU can model it. Vendor chips keep their boot ROM closed, so none has a QEMU model

> ⚠ WATCH OUT
>
> **What it can't show: QEMU loads the bootloader with no verification of its own, and emulated OTP is ordinary writable memory. The verification chain works as on hardware, but the hardware anchor is only simulated.**

Live: What We'll Show

Three scenarios:

1. **Tampered image**: flip one byte in the payload, rejected before it runs
2. **Wrong key**: sign with a key the ROTPK doesn't recognize, rejected
3. **Valid signature**: the untouched image boots normally

> **NOTE**
>
> Each rejection prints the same message. That's deliberate, because reporting *which* check failed would tell an attacker what to work around.



Demo



What We Emulate



- **The board**: Arm's MPS2 AN521, a Cortex-M33 with TrustZone, emulated by QEMU; no hardware needed
- **The boot chain**: BL2 (MCUboot-derived) → TF-M (secure world) → Zephyr app (non-secure world)
- **The flash**: the build merges every image into one file, `tfm_merged.hex`, which QEMU loads into emulated flash
- **The anchor**: the ROTPK sits in emulated OTP, which is ordinary memory here: the hardware anchor is only simulated

The Demo Tooling

Script	What it does	Why a script
`dev.sh`	Runs any command in a Docker container with Zephyr 3.7.2, TF-M and the SDK	Same result on any machine, nothing to install
`qemu.sh`	Boots one `.hex` file on the emulated AN521	The board has no power-off, so a timeout ends each run
`hexlab.py`	Changes bytes in a `.hex` file	Every HEX line has a checksum; a hand edit crashes the CPU instead of triggering the secure boot check
`demo.sh`	Runs each case end to end	Builds once, reruns quickly, the same way every time

Case 1: Tampered Image

- **What we do**: take the valid image and flip one byte inside the app's firmware
- **What happens**: BL2 recomputes the hash, it no longer matches the SHA256 stored in the image, and the boot stops: `Image in the primary slot is not valid!`
- **What it shows**: one byte is enough; flipping it back boots again

```
hexlab.py tamper build-psa/zephyr/tfm_merged.hex out/tampered.hex
qemu.sh out/tampered.hex
```

> **NOTE**
>
> The flipped byte is at `0x00102000`: 8 KB into the app's slot, past the 1 KB header, inside the firmware covered by the SHA256.



Case 2: Wrong Key



- **What we do**: generate our own key pair and build the app signed with it; the image is intact and correctly signed
- **What happens**: the image carries our new public key, it does not match the ROTPK in OTP, and BL2 rejects it
- **What it shows**: the key in OTP decides what runs; a valid signature from any other key is worthless

```
gen-keys.sh
west build -d build-keys -b mps2/an521/cpu0/ns app/upgrade_probe -- -DEXTRA_CONF_FILE=conf/own-keys.conf
qemu.sh build-keys/zephyr/tfm_merged.hex
```



Case 3: Valid Signature



- **What we do**: boot the original image, untouched and signed with the key that matches OTP
- **What happens**: BL2 checks the hash and signature, both pass, and the app runs
- **What it shows**: everything that failed before was the check doing its job

```
qemu.sh build-psa/zephyr/tfm_merged.hex
```

Thank you

Questions?





Resources



1. Zephyr: Security overview and hardening tool
2. Zephyr: user mode and memory domains
3. Zephyr: PSA Crypto
4. Zephyr: TF-M integration
5. MCUboot documentation
6. TF-M: secure boot design
7. ETSI EN 303 645 mapping
8. Zephyr: sysbuild